

React Native: Advanced Tutorial

This assumes you're comfortable building full apps with navigation, state, and API calls. Here we go under the hood: performance, native code, animations, testing, and shipping to production.

1. Understanding the Architecture

Modern React Native runs on the **New Architecture** (default since RN 0.76+):

- **JSI (JavaScript Interface)** replaces the old async bridge, letting JS and native code call each other directly and synchronously when needed.
- **Fabric** is the new rendering system — it can commit UI updates in a single pass without round-tripping through the old bridge, reducing jank.
- **TurboModules** replace the old Native Modules system with lazy loading — native modules only initialize when first used, speeding up startup.

You mostly won't touch these directly, but understanding them matters when a library claims "New Architecture support" or you're debugging why a native call feels slow.

2. Performance Optimization

Avoid unnecessary re-renders

```
import { memo, useCallback, useMemo } from
'react';

const ListItem = memo(function ListItem({ item,
onPress }) {
  console.log('rendering', item.id); // watch
this to catch wasted renders
  return (
    <Pressable onPress={() => onPress(item.id)}>
      <Text>{item.title}</Text>
    </Pressable>
  );
});

function ProductList({ products }) {
  // Without useCallback, a new function is
created every render,
  // which breaks memo() on ListItem since props
"change" every time.
  const handlePress = useCallback((id) => {
    console.log('pressed', id);
  }, []);

  // useMemo avoids recomputing expensive
derived data every render
  const sorted = useMemo(
    () => [...products].sort((a, b) => a.price -
b.price),
    [products]
  );

  return (
    <FlatList
      data={sorted}
      keyExtractor={(item) => item.id}
    >
```

```

        renderItem={({ item }) => <ListItem item=
{item} onPress={handlePress} />}
      />
    );
  }
}

```

FlatList tuning for long lists

```

<FlatList
  data={products}
  renderItem={renderItem}
  keyExtractor={(item) => item.id}
  initialNumToRender={10}          // don't render
  everything up front
  maxToRenderPerBatch={10}
  windowSize={5}                  // how many
  "screens" worth to keep mounted
  removeClippedSubviews={true}    // unmount
  offscreen items (Android especially)
  getItemLayout={(data, index) => (
    { length: ITEM_HEIGHT, offset: ITEM_HEIGHT *
  index, index }
  )}
/>

```

`getItemLayout` matters most: it lets FlatList skip measuring items, which is often the biggest perf win for uniform-height lists. For very large or complex lists, consider **FlashList** (from Shopify) as a drop-in FlatList replacement with far better recycling.

Profiling

- Use **Flipper** or the **React DevTools Profiler** to find slow components.
- Enable the in-app **Perf Monitor** (shake device → "Show Perf Monitor") to watch JS and UI frame rates live.
- Hermes (RN's default JS engine) ships with a sampling profiler — `npx react-native profile-hermes` — for finding JS-thread bottlenecks.

3. Animations with Reanimated

The `Animated` API runs on the JS thread and can drop frames under load. **Reanimated** runs animations on the UI thread directly, so they stay smooth even if JS is busy.

```
npx expo install react-native-reanimated
```

```
import { View, Pressable, Text } from 'react-native';
import Animated, {
  useSharedValue,
  useAnimatedStyle,
  withSpring,
} from 'react-native-reanimated';

function LikeButton() {
  const scale = useSharedValue(1);

  const animatedStyle = useAnimatedStyle(() => ({
    transform: [{ scale: scale.value }],
  }));

  const handlePress = () => {
```

```

    scale.value = withSpring(1.4, {}, () => {
      scale.value = withSpring(1);
    });
  };

  return (
    <Pressable onPress={handlePress}>
      <Animated.View style={[styles.heart,
animatedStyle]}>
        <Text>❤️</Text>
      </Animated.View>
    </Pressable>
  );
}

```

Key idea: `scale` is a **shared value** that lives outside React's render cycle. Mutating `.value` updates the UI thread directly — no re-render, no bridge traffic, no dropped frames.

Pair Reanimated with **react-native-gesture-handler** for swipe-to-dismiss, draggable cards, or bottom sheets — gestures also need to run off the JS thread to feel responsive.

4. State Management at Scale

`useState` and `Context` work for small apps, but `Context` re-renders every consumer on any change. For larger apps, a dedicated store avoids that.

Zustand (lightweight, minimal boilerplate)

```

import { create } from 'zustand';

```

```

const useCartStore = create((set) => ({
  items: [],
  addItem: (item) => set((state) => ({ items:
[...state.items, item] })),
  removeItem: (id) =>
    set((state) => ({ items:
state.items.filter((i) => i.id !== id) })),
  total: () => 0,
}));

// In a component – only re-renders when `items`
changes, not on unrelated state
function CartBadge() {
  const itemCount = useCartStore((state) =>
state.items.length);
  return <Text>{itemCount} items</Text>;
}

```

Redux Toolkit (larger apps, more structure/tooling)

```

import { createSlice, configureStore } from
 '@reduxjs/toolkit';

const cartSlice = createSlice({
  name: 'cart',
  initialState: { items: [] },
  reducers: {
    addItem: (state, action) => {
      state.items.push(action.payload); // Immer
lets you "mutate" safely
    },
    removeItem: (state, action) => {
      state.items = state.items.filter((i) =>
i.id !== action.payload);
    },
  },
});

```

```
});  
  
export const { addItem, removeItem } =  
  cartSlice.actions;  
export const store = configureStore({ reducer: {  
  cart: cartSlice.reducer } });
```

Rule of thumb: reach for Zustand or Redux Toolkit once Context re-renders become a measurable problem, not by default — most screens are fine with local state and one or two well-placed contexts.

5. Native Modules (Bridging to Native Code)

Sometimes you need functionality Expo/RN doesn't expose — a native SDK, custom hardware access, etc. With **Expo Modules API** (the modern approach):

```
npx create-expo-module my-native-module
```

This scaffolds Swift (iOS) and Kotlin (Android) files alongside a typed JS interface. Example iOS side:

```
// ios/MyNativeModule.swift  
import ExpoModulesCore  
  
public class MyNativeModule: Module {  
  public func definition() -> ModuleDefinition {  
    Name("MyNativeModule")  
  
    Function("getDeviceModel") { () -> String in  
      return UIDevice.current.model  
    }  
  }  
}
```

```
}  
}  
}
```

Called from JS:

```
import MyNativeModule from 'my-native-module';  
  
const model = MyNativeModule.getDeviceModel();
```

For most apps, check `reactnative.directory` first — there's likely already a maintained library before you need to write native code yourself.

6. Testing

Unit and component tests: Jest + React Native Testing Library

```
import { render, screen, fireEvent } from  
  '@testing-library/react-native';  
import Counter from './Counter';  
  
test('increments count when button is pressed',  
  () => {  
    render(<Counter />);  
    const button = screen.getByText('Add One');  
  
    fireEvent.press(button);  
  
    expect(screen.getByText('1')).toBeTruthy();  
  });
```


Test behavior (what the user sees and does), not implementation details like internal state names.

End-to-end tests: Detox or Maestro

Detox drives the real app on a simulator/emulator:

```
describe('Login flow', () => {
  it('should log in with valid credentials',
    async () => {
      await element(by.id('email-
input')).typeText('test@example.com');
      await element(by.id('password-
input')).typeText('password123');
      await element(by.id('login-button')).tap();
      await expect(element(by.text('Welcome
back!'))).toBeVisible();
    });
});
```

Maestro is a newer, YAML-based alternative that's often faster to set up:

```
appId: com.myapp
---
- launchApp
- tapOn: "Email"
- inputText: "test@example.com"
- tapOn: "Password"
- inputText: "password123"
- tapOn: "Log In"
- assertVisible: "Welcome back!"
```

7. CI/CD with EAS

Expo Application Services (EAS) handles builds and submission without you needing Xcode/Android Studio configured locally.

```
npm install -g eas-cli
eas build:configure
```

`eas.json` defines build profiles:

```
{
  "build": {
    "development": { "developmentClient": true,
    "distribution": "internal" },
    "preview": { "distribution": "internal" },
    "production": { "autoIncrement": true }
  }
}
```

```
eas build --platform all --profile production
eas submit --platform ios
```

Wire this into GitHub Actions to build on every merge to `main`, and use **EAS Update** to push JS-only fixes over-the-air without a full app store review cycle.

8. Deep Linking & Universal Links

```
// app.json (Expo)
{
  "expo": {
    "scheme": "myapp",
    "ios": { "associatedDomains":
```

```

["applinks:myapp.com"] },
  "android": {
    "intentFilters": [{
      "action": "VIEW",
      "data": [{ "scheme": "https", "host":
"myapp.com" }],
      "category": ["BROWSABLE", "DEFAULT"]
    }]
  }
}
}
}

```

React Navigation picks this up automatically if you configure linking:

```

const linking = {
  prefixes: ['myapp://', 'https://myapp.com'],
  config: {
    screens: {
      Home: 'home',
      ProductDetail: 'product/:id', //
myapp://product/42 or
https://myapp.com/product/42
    },
  },
};

<NavigationContainer linking={linking}>
  { /* ... */ }
</NavigationContainer>

```

9. Offline Support

Combine a persistent store with a sync strategy:

```

import AsyncStorage from '@react-native-async-storage/async-storage';
import NetInfo from '@react-native-community/netinfo';

// Queue writes locally when offline, flush when
connectivity returns
async function saveTodo(todo) {
  const queue = JSON.parse((await
AsyncStorage.getItem('pendingWrites')) ?? '[]');
  queue.push(todo);
  await AsyncStorage.setItem('pendingWrites',
JSON.stringify(queue));

  const state = await NetInfo.fetch();
  if (state.isConnected) {
    await flushQueue();
  }
}

NetInfo.addEventListener((state) => {
  if (state.isConnected) flushQueue();
});

```

For anything beyond simple queuing, look at **WatermelonDB** (reactive local database built for RN, syncs well with backends) or **TanStack Query**'s offline/retry features paired with its cache persistence.

10. Security Essentials

- **Never store secrets in JS bundles** — they're extractable from the compiled app. Use a backend to broker access to third-party APIs.

- **Store sensitive data in Keychain/Keystore**, not `AsyncStorage` (which is unencrypted). Use `expo-secure-store` or `react-native-keychain`.
- **Certificate pinning** for high-security apps (banking, health) via libraries like `react-native-ssl-pinning`.
- **Validate deep link params** before using them — treat them as untrusted input, same as URL query params on the web.
- **Enable Hermes + code obfuscation** (e.g., via ProGuard on Android) to raise the bar against reverse engineering, though nothing makes a client app fully tamper-proof.

11. Background Tasks & Notifications

```
npx expo install expo-notifications expo-background-task
```

```
import * as Notifications from 'expo-notifications';

Notifications.setNotificationHandler({
  handleNotification: async () => ({
    shouldShowAlert: true,
    shouldPlaySound: true,
    shouldSetBadge: false,
  }),
});

async function registerForPushNotifications() {
  const { status } = await
  Notifications.requestPermissionsAsync();
  if (status !== 'granted') return null;
```

```
const token = await
Notifications.getExpoPushTokenAsync();
return token.data; // send this to your
backend to target this device
}
```

Background tasks (syncing data periodically, even when the app is closed) are heavily constrained by both iOS and Android for battery reasons — expect them to run on the OS's schedule, not yours, and design for "eventually" rather than "precisely on time."

Where to Go From Here

- Read through the **New Architecture** working group docs on GitHub to track what's changing under the hood.
- Study a production-grade open source RN app (e.g., Expo's own example apps) to see these patterns combined at scale.
- If you're building for a large team, invest early in a strong **design system** (shared components, theming) and **CI checks** (type checking, lint, tests) — the cost of skipping this compounds fast.